

☐ L→4.1 Compare ArrayList, Vector, LinkedList

=> Comparison between ArrayList, Vector, LinkedList

Feature	ArrayList	Vector	LinkedList
Underlying structure	Dynamic Array	Dynamic Array	Doubly Linked List
Synchronization	Not Synchronized	Synchronized	Not Synchronized
Random Access	Fast - $O(1)$	Fast $O(1)$	Slow $O(n)$
Insertion/Deletion	Slow in middle because elements shift $O(n)$	Slow in middle due to shifting $O(n)$	Fast at known position because links are changed $O(1)$
Performance	Generally faster than Vector	Slower due to synchronization overhead	Better for frequent insert/delete operations.
In Short	Best for frequent searching / accessing	Similar to ArrayList but synchronized	Best when frequent insertion / deletion required

Q L-→ 4.2 : Explain Set and its implementation

HashSet, LinkedHashSet, TreeSet. How does TreeSet maintain ordering

=> A set is a collection in Java's Collection

Framework that does not allow duplicate elements. It models the mathematical concept of a set and is used when uniqueness of elements matters more than order based access.

Implementation :

HashSet :

Stores unique elements using a hash table. Does not maintain insertion order. Generally provides fast insertion, deletion and searching.

LinkedList :

Stores unique element using hash table + linked list. Maintains the insertion order of elements provides approximately $O(1)$ average performance for basic operation.

TreeSet :

Stores unique elements in sorted order. It is generally implemented using a Red Black Tree. Basic insertion, deletion and searching take $O(\log n)$.

How does TreeSet maintain order:

TreeSet internally uses a self balancing Red Black Tree. When an element is inserted, it is placed according to its natural ordering or supplied Comparator. Therefore, elements are automatically kept in sorted order.

Ex 1 -> 4.3 stores 5 student names in an
ArrayList and in TreeSet.

=> Code:

```
import java.util.* ;  
public class Main {  
    public static void main (String[] args) {  
        ArrayList<String> students = new ArrayList<>();  
        students.add("Obayed");  
        students.add("Khalid");  
        students.add("Raiyyan");  
        students.add("Jubayed");  
        students.add("Jahid");  
        System.out.println("ArrayList:");  
        for (String name : students) {  
            System.out.println(name);  
        }  
  
        TreeSet<String> sortedStudents = new  
            TreeSet<>(students);  
        for (String name : sortedStudents) {  
            System.out.println(name);  
        }  
    }  
}
```

Sample Output:

ArrayList: Ob

Obayet

Khalid

Raiyyan

Jubayer

Jahid

TreeSet:

Jahid

Jubayer

Khalid

Raiyyan

Obayet.

Q L-5.1 :

Ways to Implement Multithreading in Java:

1. Extending Thread Class:

- Create a class that extends Thread.
- Override the run() method.
- Call start() to begin the thread.

Example :

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}
```

2. Implementing Runnable:

- Create a class that implements Runnable.
- Implement the run() method and pass the object to a thread.
- This is more flexible because the class can extend another class.

Example:

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Task running");  
    }  
}
```

3. Using ExecutorService

Example:

```
ExecutorService service = Executors.newFixedThreadPool(2);
```

```
Future<Integer> result = service.submit(() -> 10);  
service.shutdown();
```

Preferred:

ExecutorService is generally preferred for larger applications because

Q L → 5.2 :

Two Threads in Java:

```
class MyThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread: " + i);  
        }  
    }  
}
```

```
class MyRunnable implements Runnable {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Runnable: " + i);  
        }  
    }  
}
```

```
public class Main {  
    public static void main (String[] args) {  
        MyThread T1 = new MyThread();  
        MyRunnable T2 = new MyRunnable();  
        Thread t2 = new Thread(T2);  
    }  
}
```

```
t1.start();
```

```
t2.start();
```

```
}
```

```
}
```

Sample Output :

Thread : 1

Runnable : 1

Thread : 2

Runnable : 2

Thread : 3

Runnable : 3

Thread : 4

Runnable : 4

Thread : 5

Runnable : 5

Ex L-5.3: Custom checked exception
InvalidRadiusException. Circle constructor throws
it if radius < 0 . Main reads radius, handles
via try catch, else prints area.

=> Custom checked Exception in Java:

```
import java.util.Scanner;  
  
class InvalidRadiusException extends Exception {  
    public InvalidRadiusException(String message) {  
        super(message);  
    }  
}  
  
class Circle {  
    private double radius;  
  
    Circle(double radius) throws InvalidRadiusException {  
        if (radius < 0) {  
            throw new InvalidRadiusException("Radius cannot be negative");  
        }  
        this.radius = radius;  
    }  
}
```

```

double area;
return Math.PI * radius * radius;
}
}

public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter radius: ");
        double radius = sc.nextDouble();
        try {
            Circle c = new Circle (radius);
            System.out.println ("Area = " + c.area);
        } catch (InvalidRadiusException e) {
            System.out.println ("Error: " + e.getMessage());
        }
    }
}

```

Sample Output:

For valid radius:

Enter radius: 5

Area = 78.5398163397

For invalid radius:

Enter radius: -3

Error: Radius cannot be negative.